Rafeeq Analytics

Frontend Architecture & Data-Flow Reference

Next.js 16 (App Router) · React 19 · TypeScript · Tailwind v4 · NextAuth · Recharts
Customer-experience intelligence dashboard for a Qatar food-delivery operation

Technical documentation for engineers and external reviewers

Rafeeq Analytics — Frontend

A Next.js dashboard that visualises the backend's three intelligence pillars. It is a thin presentation layer: all heavy computation happens server-side, the frontend fetches JSON, scopes it to a time window, and renders charts, tables, maps, and a conversational assistant — fully bilingual (English / Arabic with RTL).

0. Table of Contents

1. Overview & Tech Stack
2. App Architecture & Provider Tree
3. Authentication & Roles
4. Global State (Contexts)
5. Data Layer (api.ts)
6. The Universal Page Data-Flow Pattern
7. Internationalisation & RTL
8. Page-by-Page Reference
9. Component Catalogue
10. Frontend → Backend Endpoint Map

1. Overview & Tech Stack

Concern	Technology
Framework	Next.js 16 (App Router), React 19, TypeScript
Styling	Tailwind CSS v4, `tw-animate-css`, shadcn-style primitives, class-variance-authority
Auth	NextAuth v5 (beta) — Google provider + placeholder credentials
Charts	Recharts 3
Maps	Leaflet + react-leaflet (Qatar call-origin map)
Markdown	react-markdown + remark-gfm (chatbot + drivers report)
Theming	next-themes (light / dark / system)
Icons	lucide-react
Fonts	Geist (sans/mono), Archivo (display) via next/font/google; Cairo (Arabic) self-hosted via next/font/local

The API base is configurable via `NEXT_PUBLIC_API_URL` (defaults to `http://localhost:8000`).

2. App Architecture & Provider Tree

The App Router structure: a single root layout wraps every route in a stack of client-side providers. `app/layout.tsx`:

```
<html> (font variables: Geist, Archivo, Cairo)
 └ SessionProvider                (NextAuth session)
    └ ThemeProvider               (next-themes — light/dark/system)
       └ SettingsProvider         (auto-refresh cadence, SLAs, thresholds, language)
          └ TimeFilterProvider    (app-wide WTD/MTD/QTD/YTD/All window)
             └ MessageStatusProvider  (client-side flagged/resolved state)
                └ NotificationsProvider (role-aware derived alerts)
                   └ {page}
```

Routes

Route	Page	Pillar
`/`	Call Intelligence (home)	01
`/cx-dashboard`	CX Analytics Dashboard (cross-pillar)	all
`/messages`	Support Messages	02
`/cancellations`	Cancellation Intelligence	03
`/chatbot`	Conversational assistant	assistant
`/settings`	App settings	—
`/help`	Help & glossary	—
`/login`	Sign-in	—
`/api/auth/[...nextauth]`	NextAuth route handler	—

Every dashboard shares the same chrome: a `Sidebar` (navigation) and a `Topbar` (search, time-range, notifications bell, account menu, theme/language toggles).

3. Authentication & Roles

Auth is NextAuth v5, split across `auth.config.ts` (edge-safe config + route gate) and `auth.ts` (providers + callbacks).

- **Google provider** — only enabled once `AUTH_GOOGLE_ID` exists. A hard domain gate rejects any verified email not on `@gorafeeq.com`.
- **Placeholder credentials provider** — the current live login: any `@gorafeeq.com` email plus a shared `DEV_LOGIN_PASSWORD` (default `rafeeq`). Explicitly insecure, a stand-in until Google OAuth is provisioned.
- **Legacy full bypass** — `NEXT_PUBLIC_AUTH_BYPASS=true` disables the login screen entirely; off by default, superseded by the placeholder login.

> The route gate lives in the `authorized()` callback: unauthenticated users are redirected to `/login`; logged-in users hitting `/login` bounce to the app.

Roles

A **role** (`employee` | `manager`) is chosen at login, persisted into the JWT, and exposed on the session (`lib/roles.ts` , read via `useRole()`). It drives which notifications a user receives: employees get SLA + cancellation alerts; managers additionally get agent-helpfulness alerts (which name the agent).

4. Global State (Contexts)

Four React contexts in `lib/` hold app-wide state. All persist to `localStorage` and sync across browser tabs via the `storage` event.

Context	Holds	Drives
`settings-context`	refresh cadence, chat/general SLA hours, cancel & sentiment-spike thresholds, alert toggles, confidence %, language	auto-refresh, threshold alerts, notification filtering, RTL flip. Also exports `useAutoRefresh(cb)` , a shared interval hook.
`time-filter-context`	the selected range (WTD/MTD/QTD/YTD/All) + derived `{start,end}` params + a stable `queryKey`	every dashboard's window — both client filtering and server re-queries
`message-status-context`	per-message `flagged` / `resolved` flags (client-only; no backend mutation endpoint yet)	message table state; resolving clears the SLA notification
`notifications-context`	role-aware notifications *derived* from live data	the top-bar bell panel

How notifications are derived

The notifications provider does not call a notifications endpoint — it re-derives alerts from the same dashboard data on the same auto-refresh cadence: SLA breaches from `fetchMessages()` (handling time vs SLA target), high-risk cancellations from `fetchLiveQueue()` (orders flagged "high"), and poor helpfulness from `fetchCalls()` (manager only). Read/dismissed state is persisted with stable per-alert ids.

5. Data Layer (lib/api.ts)

`lib/api.ts` is the single typed gateway to the backend. Two primitives wrap `fetch` :

```
const BASE = process.env.NEXT_PUBLIC_API_URL ?? "http://localhost:8000"
apiFetch(path)        // GET, { cache: "no-store" }, returns null on any error
apiPost(path, body)   // POST JSON, { cache: "no-store" }, returns null on error
```

No client cache: every request uses `cache: "no-store"` , so the browser never serves stale data. Caching is the backend's job (its 5-minute TTL caches). Errors return `null` rather than throwing, so a failed panel keeps its previous data instead of crashing the page.

Notable fetchers

Function	Hits	Notes
`fetchCalls()`	`/calls`	Pages through up to 1000 calls (200/page), maps to `CallRecord`
`fetchMessages()`	`/api/v1/sentiment/results`	Pages up to 1000; maps API shape → `SupportMessage`, derives time-of-day + handling minutes
`fetchAllMessagesData(range)`	5 endpoints in parallel	Raw feed (client-filtered) + pre-aggregated panels (server-windowed via `?start&end`)
`fetchAllCancellationData(range)`	11 endpoints in parallel	All cancellation analytics + model info + live queue; drivers report fetched separately (slow)
`fetchLiveQueue`, `explainOrder`, `askCancellationChat`	/api/cancellation/*	Live scoring, per-order explanation, Q&A

6. The Universal Page Data-Flow Pattern

Every dashboard page follows the same lifecycle, so understanding one explains them all:

```
1. Mount        → loadData(range, background=false)   shows full skeleton
2. Range change → effect on timeFilter.queryKey → loadData(range, true)   background refresh
3. Auto-refresh → useAutoRefresh(() => loadData(range, true))   on the Settings cadence
4. Manual       → RefreshStatus "Refresh" button → loadData(range, true)

Two kinds of data, two scoping strategies:
  • Record lists (calls, messages)  → fetched in full, filtered CLIENT-SIDE via
       filterByRange() so toggling the range re-scopes instantly.
  • Pre-aggregated panels (trends, heatmaps, cancellation analytics) → re-queried
       SERVER-SIDE with ?start&end for the selected window.
```

The selected window comes from `useTimeFilter()`; the cadence from `useAutoRefresh()` (default 120s, configurable 30s–10min or Off in Settings). "background" loads skip the skeleton so the UI doesn't flash.

7. Internationalisation & RTL

`lib/i18n.tsx` is a lightweight dictionary keyed by semantic strings (`nav.*`, `cx.*`, `cancel.*`, …), each with `en` + `ar` values. Usage:

```
const t = useT()              // t("nav.settings") → "Settings" | "الإعدادات"
t("set.slaBreachDesc", { chat: 4 })   // interpolates {chat}
const tv = useTV()            // value-map translator for dynamic data labels
```

The active language lives in `settings.language`. Selecting Arabic flips the whole document to RTL (`SettingsProvider` sets `<html dir="rtl" lang="ar">`). Arabic renders in the self-hosted Cairo typeface;

charts are forced back to LTR so Recharts geometry stays correct.

8. Page-by-Page Reference

8.1 Call Intelligence — `/` (Pillar 01)

The home dashboard. Loads existing calls via `fetchCalls()`; calls carry a per-record `datetime` so the entire page (stat cards, charts, Qatar map, table) derives from one client-filtered set (`scopedCalls`).

- **Upload & analyse:** users drop `.txt/.csv/.json` transcripts; the page parses them client-side and POSTs to `/analyse/batch` in chunks of 100, prepending results to the list.
- **Renders:** 4 stat cards, `FilterBar` (category/sentiment/agent + CSV export), `QatarMap` (Leaflet call-origin map), `CallTable`, `SentimentTrend`, `ToneChart`, `IntentPanel` (click-to-filter), `CategoryBreakdown`, `TopTopics`.

8.2 CX Dashboard — `/cx-dashboard` (cross-pillar)

The single-pane health view. `loadData` fetches from *six* sources in parallel — calls, all message data, cancellation trend, cancellation by-zone, feature importance, cancellation by-actor — blending all three pillars. Calls/messages are client-scoped; cancellation aggregates are server-windowed.

Renders top-line KPIs (interactions, sentiment, cancellation rate, resolution, bot containment, escalation), a weekly CX health score, top complaints, text sentiment, and cancellation drivers. Resolution-time and chatbot-telemetry panels are behind a "Work in progress" overlay (no backing data source yet) using mock previews.

8.3 Support Messages — `/messages` (Pillar 02)

Uses `fetchAllMessagesData(range)`: the raw feed is filtered client-side; `TopNegativeTriggers`, `CrossChannelComparison`, `SentimentByZoneTime`, and `MessageSentimentTrend` are re-queried server-side per window. Deep-linkable: `/messages?msg=MSG-XXXX` (from a notification) opens that message's detail modal.

- **Threshold alerts:** a sentiment-spike alert (negative jump week-over-week past `sentimentSpikePct`) and an SLA-breach indicator (handling time = `closed_at − created_at` vs the channel's SLA target).
- Flag/resolve actions write to `message-status-context` (local only).

8.4 Cancellations — `/cancellations` (Pillar 03)

`fetchAllCancellationData(range)` loads every analytics panel + model info + the live high-risk queue; the Gemini drivers report is fetched separately so the charts paint first. A **prediction-engine selector** (`auto` / `model` / `gemini`) controls live-queue scoring.

- **Renders:** stat cards, weekly cancel-rate line, drivers (model + Gemini report), by-actor, zone×time heatmap, by-zone, by-day, the live high-risk queue (per-order "Explain" calls `explainOrder()`), model-performance card, and an "Ask the Cancellation Analyst" chat (`askCancellationChat()`).
- **Cancellation-rate alert:** zones above `cancelThresholdPct` with meaningful volume.

8.5 Chatbot — `/chatbot`

A conversational UI that POSTs the full message history to `/chat`. Supports image attachments (read as base64) and browser speech-to-text. Bot replies render as GitHub-flavoured Markdown (tables, headings, blockquotes).

8.6 Settings — `/settings`

The control panel that writes to `settings-context`: appearance (theme + language), auto-refresh interval, SLA targets, alert thresholds + channels, notification toggles, ML confidence threshold. Some sections (keyword→intent simulator, external integrations) are flagged work-in-progress.

8.7 Help & Login

Help — metrics glossary, ML-classifier guide, FAQ, and an internal feedback form. **Login** — the placeholder credentials form (work email + shared password + role select), or the Google button once OAuth is live.

9. Component Catalogue

Reusable presentation components live in `components/rafeeq/`. Highlights:

Group	Components
Chrome	`sidebar`, `topbar`, `notification-panel`, `refresh-status`, `time-range-select`, `theme-toggle`, `logo`, `brand-badge`
Calls (P1)	`call-table`, `call-detail-modal`, `sentiment-trend`, `tone-chart`, `intent-panel`, `category-breakdown`, `top-topics`, `qatar-map-leaflet`, `hero-banner`, `filter-bar`
Messages (P2)	`message-feed-table`, `message-detail-modal`, `top-negative-triggers`, `cross-channel-comparison`, `sentiment-by-zone-time`, `message-sentiment-trend`
Cancellations (P3)	`cancellation-chat`
Shared UI	`stat-card`, `panel`, `dropdown`, `column-filter`, `threshold-alert`, `loading-screen`, `work-in-progress`, `role-select`

Mock data modules (`lib/mock-*.ts`) back the work-in-progress panels and provide types; real data always comes from `api.ts`.

10. Frontend → Backend Endpoint Map

Page / Feature	Frontend call	Backend endpoint(s)
Call Intelligence list	`fetchCalls()`	`GET /calls`
Call upload/analyse	`fetch /analyse/batch`	`POST /analyse/batch`
Support Messages	`fetchAllMessagesData()`	`/api/v1/sentiment/results` + `/api/v1/analytics/{top-negative-triggers,cross-channel,sentiment-trend,zone-heatmap}`
Cancellations	`fetchAllCancellationData()`	`/api/cancellation/analytics/*`, `/model/info`, `/predict/live-queue`
Cancellation explain / chat	`explainOrder()`, `askCancellationChat()`	`POST /api/cancellation/explain/{id}`, `POST /api/cancellation/chat`
Drivers report	`fetchDriversReport()`	`GET /api/cancellation/analytics/drivers-report`
Chatbot	`fetch /chat`	`POST /chat`
CX Dashboard	6 parallel fetchers	calls + messages analytics + cancellation trend/zone/actor + feature-importance
Notifications bell	`fetchMessages`, `fetchLiveQueue`, `fetchCalls`	derived client-side from those endpoints